

1^η Σειρά Ασκήσεων Προηγμένες Εφαρμογές Ψηφιακής Σχεδίασης

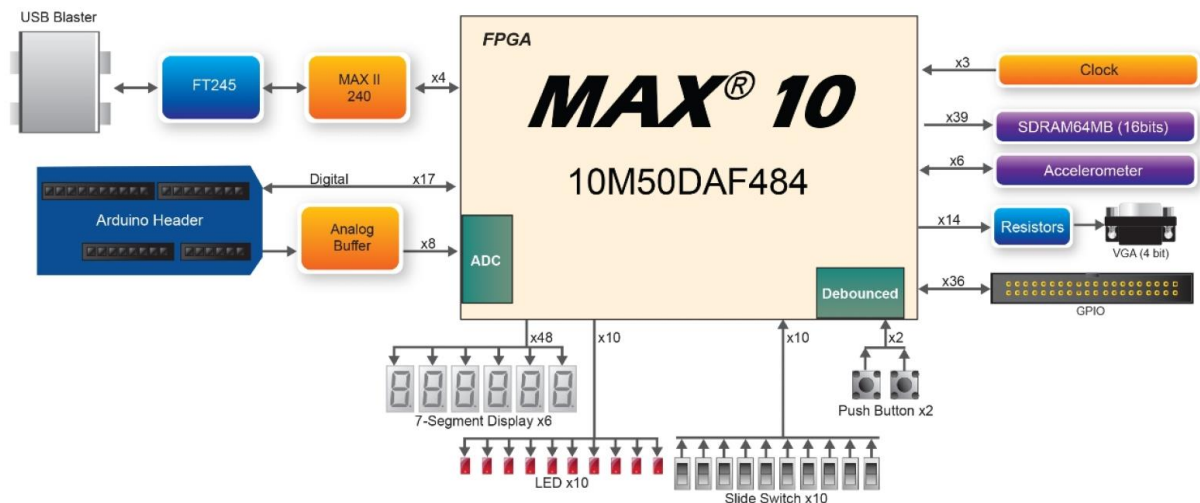
Ημερομηνία Παράδοσης: **1/12/2021 23.00 μ.μ.**

Σύστημα Παράδοσης: **courses.cs.ihu.gr**

Μορφή αρχείου: **AEM.pdf**

Σκοπός Σειράς Ασκήσεων:

Ο σκοπός αυτής της σειράς ασκήσεων είναι οι φοιτητές να περιγράψουν σχεδιάσεις, που γνωρίζουν από το μάθημα Ψηφιακής Σχεδίασης, σε **SystemVerilog** και με στοχευμένη τεχνολογία **FPGA/CPLD MAX 10 (10M50DAF484C6GES)**.



Οι φοιτητές θα υλοποιήσουν διάφορα συνδυαστικά κυκλώματα, όπως: πολυπλέκτες, αθροιστές, 8-bit ALU και άλλα, με διάφορες τεχνικές περιγραφής υλικού (HDL).

Επιπρόσθετα, οι φοιτητές θα προσομοιώσουν σχεδιάσεις SystemVerilog σε **επίπεδο RTL (Register Transfer Level)**.

Οι ασκήσεις θα υλοποιηθούν με το λογισμικό **Quartus Prime** και τον προσομοιωτή **ModelSim** ώστε οι φοιτητές να εξοικειωθούν με τις σύγχρονες ροές ψηφιακής σχεδίασης (Digital Design Flows).

Προσοχή: Η SystemVerilog είναι case-sensitive!!!

Ορίστε στο Quartus Prime τη SystemVerilog

Στο κεντρικό μενού:

Assignments > Settings > Compiler Settings > Verilog HDL Input
και επιλέξτε "**SystemVerilog**" στην κατηγορία Verilog version.

Μέρος I

Να περιγράψετε μια λογική πύλη nand τεσσάρων εισόδων με SystemVerilog και να την προσομοιώσετε μέσα από το Quartus Prime με τον προσομοιωτή ModelSim-Intel FPGA Edition. Να προσομοιωθεί η πύλη για τον πίνακα αλήθειας της.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (nand41.sv)** και συμπεριλάβετε τη στο έργο σας (με χρήση της **assign**).
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα αρχείο **.do (Save Transcript As... nand41.do)** και να εκτελέσετε ξανά την προσομοίωση (**do nand41.do**).
6. Παραδοτέα: κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Παράδειγμα εντολών command line **Altera-ModelSim**:

```
-- enter simulation
vsim nand41

-- add ALL signals to the wave window
-- add wave *

-- setup input signals
add wave -divider Inputs
add wave -height 30 -color #6F1AD1 a -color #317523 b -color #E6D706
c -color #DEE2CA d

-- setup input signals
force -repeat 20 a 0 0, 1 10
force b 0 0, 1 20, 0 40, 1 100, 0 150
force -repeat 40 c 0 0, 1 20
force d 0 0, 1 20, 0 40, 1 200, 0 350

-- setup output signals
add wave -divider Outputs
add wave -height 30 -color #FD0707 y

-- run the simulation
run 200
wave zoom full

-- restart simulation
restart

-- finish simulation
quit -sim
-- quit ModelSim
quit -f
```

Στη συνέχεια από το σύμφωνο με τον **Πίνακα 3** του Παραρτήματος να υλοποιήσετε ένα συνδυαστικό κύκλωμα που θα έχει εισόδους **C, N, V, Z** (τις σημαίες – **flags** – που παράγει ένας επεξεργαστής όταν εκτελεί αριθμητικές εντολές και αποθηκεύονται στον Καταχωρητή Κατάστασης) και ως εξόδους τα **HS, LS, HI, LO, GE, LE, GT** και **LT**, που είναι απαραίτητα για προσημασμένη ή μη-προσημασμένη σύγκριση δύο αριθμών A και B. Η υλοποίηση να γίνει με **δομικό τρόπο** και με χρήση των **built-in πυλών (not, and, or, xor, xnor)** που διαθέτει η **SystemVerilog**.

Για παράδειγμα η κλήση μιας built-in πύλης:

and(output, input1, input2, ..., inputk);

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

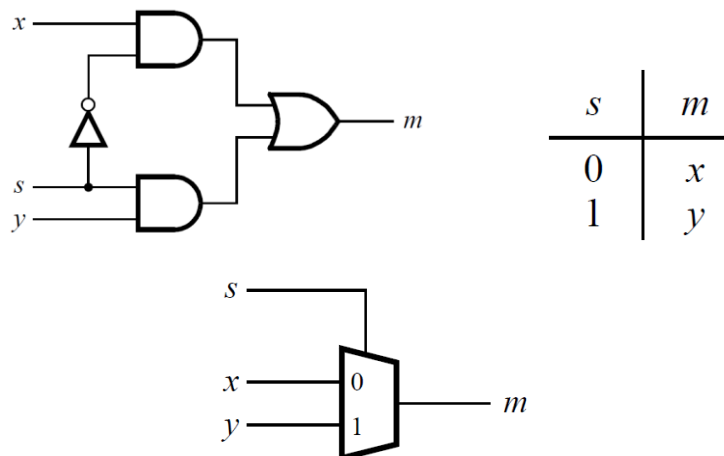
1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (CompFlags.sv)** και συμπεριλάβετε τη στο έργο σας (με χρήση της **assign** και **built-in** πύλες).
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα **αρχείο .do (CompFlags.do)** και να εκτελέσετε ξανά την προσομοίωση (**do CompFlags.do**).
6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Μέρος II

Η παρακάτω **Εικόνα 1** δείχνει ένα κύκλωμα αθροίσματος γινομένων (**Sum of Products - SOP**) το οποίο υλοποιεί ένα **πολυπλέκτη 2 προς 1 ενός bit** με είσοδο επιλογής **s**.

Αν το **s = 0** : η έξοδος του πολυπλέκτη **m** είναι ίση με την είσοδο **x**, και αν **s = 1** : η έξοδος είναι ίση με το **y**.

Επίσης, παρουσιάζεται ο πίνακας αληθείας για αυτόν τον πολυπλέκτη και το σύμβολο του κυκλώματος του.



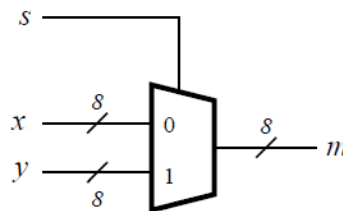
Εικόνα 1. Ένας πολυπλέκτης 2 σε 1.

Να περιγράψετε με τον τελεστή υπό συνθήκη (?) τον πολυπλέκτη 2 σε 1 ενός bit (**mux2.sv**) με **SystemVerilog**.

$$m = s ? y : x ;$$

Στη συνέχεια να περιγράψετε μια οντότητα σε **SystemVerilog** το σύστημα που περιγράφεται στην **Εικόνα 2** που έχει δύο εισόδους οκτώ δυαδικών ψηφίων, x και y και παράγει την έξοδο οκτώ δυαδικών ψηφίων m . Αν $s = 0$ τότε $m = x$, ενώ αν $s = 1$ τότε $m = y$.

Αναφερόμαστε σε αυτό το κύκλωμα ως πολυπλέκτη 2-προς-1 οκτώ δυαδικών ψηφίων. Έχει το σύμβολο κυκλώματος που φαίνεται στην Εικόνα 2, στο οποίο τα x , y και m απεικονίζονται ως σύρματα οκτώ δυαδικών ψηφίων.



Εικόνα 2. Το σύμβολο ενός πολυπλέκτη 2-προς-1 οκτώ δυαδικών ψηφίων.

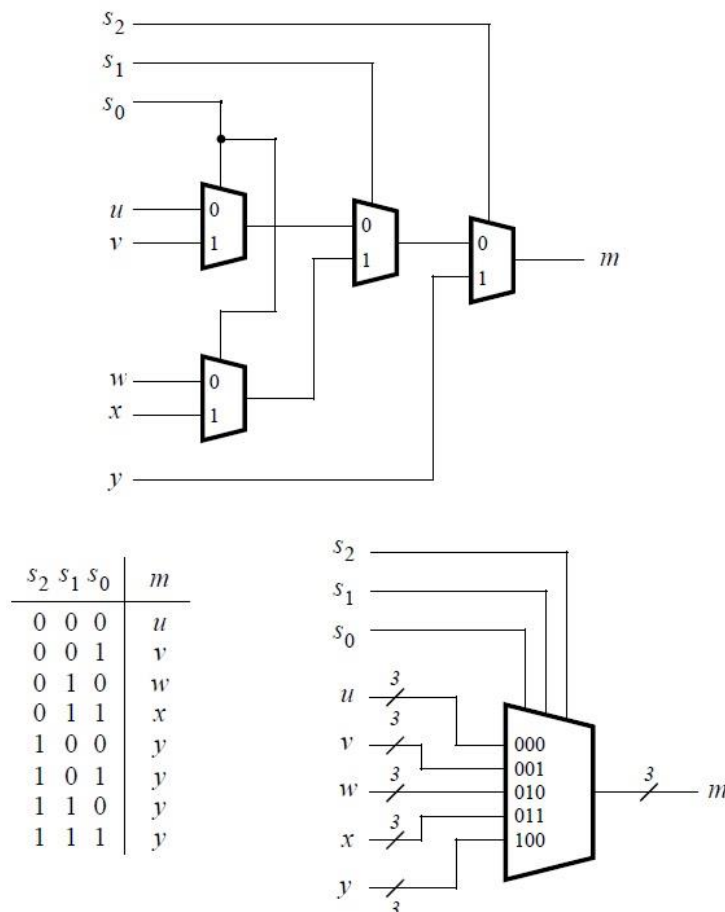
Εκτελέστε τα βήματα που προτείνονται παρακάτω:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog** (πολυπλέκτη 2-προς-1 οκτώ δυαδικών ψηφίων, **mux28bit.sv**) και συμπεριλάβετε τη στο έργο σας (με χρήση της **assign**).
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα αρχείο **.do** (**mux28bit.do**).
6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο **.do**, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Μέρος III

Στην **Εικόνα 1** παρουσιάσαμε ένα πολυπλέκτη 2 προς 1 που επιλέγει μεταξύ των δύο εισόδων x και y . Για αυτό το μέρος εξετάστε το κύκλωμα στο οποίο η έξοδος m πρέπει να επιλεγεί από πέντε εισόδους u, v, w, x και y . Στην **Εικόνα 3** παρουσιάζεται πώς μπορούμε να κατασκευάσουμε τον απαιτούμενο **πολυπλέκτη 5 προς 1 τριών (3) δυαδικών ψηφίων** χρησιμοποιώντας τέσσερις πολυπλέκτες 2 προς 1 τριών (3) δυαδικών ψηφίων. Το κύκλωμα χρησιμοποιεί είσοδο επιλογής 3-bit ($s_2s_1s_0$) και υλοποιεί τον πίνακα αληθείας που φαίνεται στην **Εικόνα 3**. Ένα σύμβολο κυκλώματος για αυτόν τον πολυπλέκτη δίνεται παρακάτω.

Στην άσκηση αυτή θα συνδυάσουμε την περιγραφή ροής δεδομένων με αυτή του δομικού τρόπου σχεδίασης. Αρχικά, περιγράψτε σε **SystemVerilog** ένα πολυπλέκτη 2 σε 1 με εισόδους τριών bit (mux23). Στη συνέχεια με δομικό τρόπο (structural) καλέστε ως υποκύκλωμα (module/subcircuit/component) τον mux23 σε τέσσερα στιγμιότυπα (instances) για να υλοποιήσετε το κύκλωμα του 3bit πολυπλέκτη 5 προς 1 (mux53 – Top-level Entity). Προσοχή στην σωστή δήλωση της ιεραρχίας των περιγραφών **SystemVerilog**.



Εικόνα 3. Η δομή και το σύμβολο ενός πολυπλέκτη 5-προς-1 τριών δυαδικών ψηφίων .

Εκτελέστε τα παρακάτω βήματα για να εφαρμόσετε τον πολυπλέκτη 5 σε 1.

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε δύο περιγραφές **SystemVerilog** την **mux23** και την mux53 (top-level entity) και συμπεριλάβετε αυτές στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα αρχείο .do (**mux53.do**).
6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

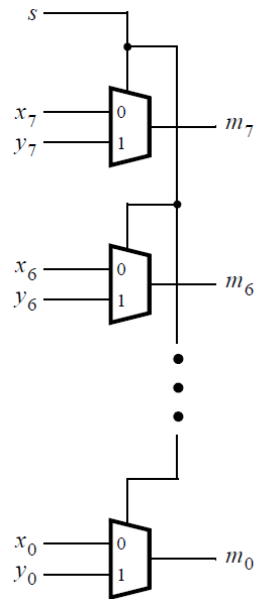
Μέρος IV

Στο μέρος αυτό θα περιγράψουμε τον **3bit πολυπλέκτη 5 προς 1** χρησιμοποιώντας την τεχνική της περιγραφής συμπεριφοράς (Behavioral) με τη δήλωση **always_comb** και την εντολή **if...elseif...else**. Το κύκλωμα θα έχει τις ίδιες εισόδους και εξόδους όπως στο **Μέρος III**.

Εκτελέστε τα βήματα που προτείνονται παρακάτω:

- 
1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
 2. Δημιουργήστε την περιγραφή **SystemVerilog mux53beh** (top-level entity).
 3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
 4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
 5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα αρχείο .do (**mux53beh.do**).
 6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Στη συνέχεια, σύμφωνα και με την **Εικόνα 4**, να υλοποιηθεί με χρήση του **Μέρους II** και της εντολής **generate** (forloop) ένας πολυπλέκτης 2-1 8 bit **mux28fg** όπου θα κληθεί το στιγμιότυπο του **mux2**.



Εικόνα 4. Η δομή ενός πολυπλέκτη 2-προς-1 οκτώ δυαδικών ψηφίων.

Εκτελέστε τα βήματα που προτείνονται παρακάτω:

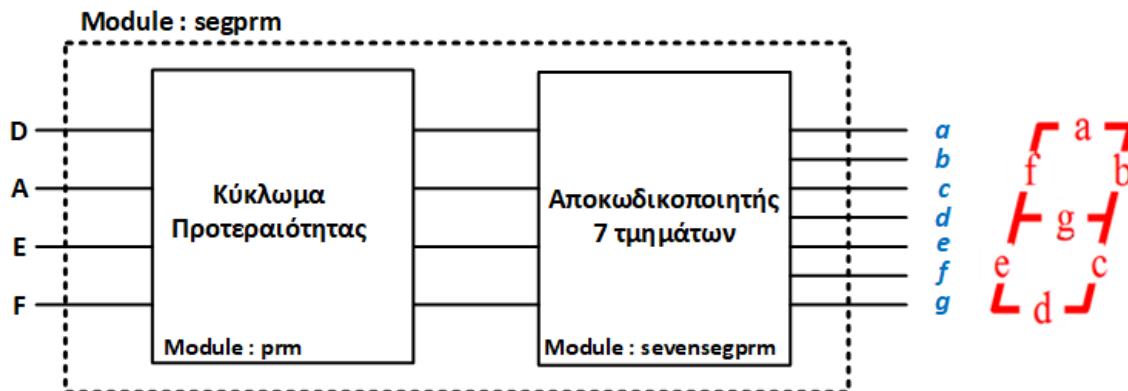
1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog mux28fg** (πολυπλέκτη 2-προς-1 οκτώ δυαδικών ψηφίων) και συμπεριλάβετε τη στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα αρχείο **.do (mux28fg.do)**.
6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο **.do**, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Μέρος V

Να υλοποιήσετε ένα **κύκλωμα προτεραιότητας** που θα εξυπηρετεί τις ανάγκες ενός δήμου για τη διάθεση του μοναδικού υπηρεσιακού οχήματος. Οι δυνητικοί οδηγοί του οχήματος είναι: Ο Δήμαρχος(D), Ο Αντιδήμαρχος(A), Ο Προϊστάμενος Διεύθυνσης(E) και ο Υπάλληλος(F) με τη σειρά προτεραιότητας που αναφέρθηκαν. Το σύστημα έχει τέσσερις εισόδους (D,A,E,F) και τέσσερις εξόδους (y[3..0]). Κάθε μέρα ο κάθε χρήστης κάνει το αίτημα του και το σύστημα δίνει πρόσβαση στο όχημα κατά προτεραιότητα. Η περιγραφή **prm sv** να γίνει με **always_comb** και χρήση της δομής **if...elseif...else**.

Στη συνέχεια να υλοποιήσετε ένα **κύκλωμα αποκωδικοποιητή** που θα έχει ως είσοδο έναν δυαδικό αριθμό 4-bit (y[3..0] από το κύκλωμα προτεραιότητας) και θα ενεργοποιεί σε ενδείκτη επτά τμημάτων τον αντίστοιχο **δεκαεξαδικό** (A,b,C,d,E,F). Η περιγραφή **sevensegprm sv** να γίνει με **always_comb** και χρήση της δομής **case**. Οι διάφορες αναπτυξιακές πλακέτες (όπως η DE10 ή DE2) διαθέτουν ενδείκτες επτά τμημάτων (κοινής ανόδου ή καθόδου). Οι ενδείκτες είναι τοποθετημένοι έτσι ώστε να είναι δυνατή η αναπαράσταση διάφορων αριθμών. Οι ενδείκτες είναι συνδεδεμένοι με

το FPGA που διαθέτει η πλακέτα. Εάν εφαρμόσουμε χαμηλό λογικό επίπεδο σε ένα τμήμα (segment) του ενδείκτη αυτό θα ανάψει (είναι κοινής καθόδου) και όταν εφαρμόσουμε υψηλό επίπεδο λογικής αυτό σβήνει.



Εικόνα 5. Το συνολικό σύστημα.

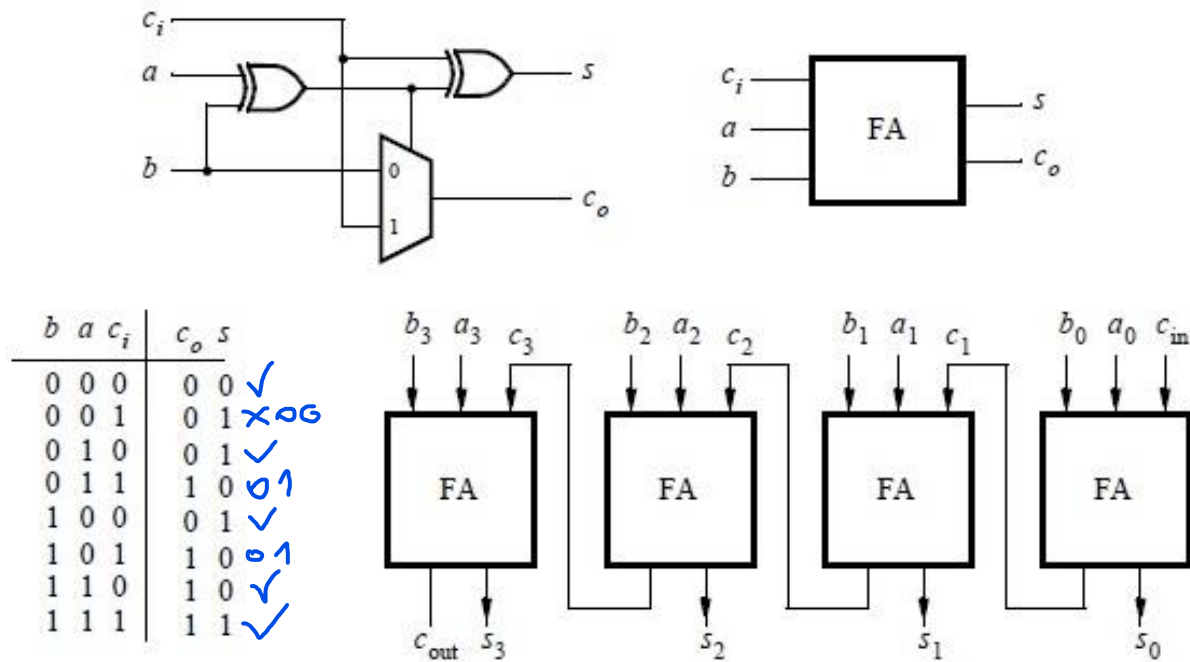
```
"1000" => d  
"0100" => A  
"0010" => E  
"0001" => F
```

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε Με δομικό τρόπο την περιγραφή **SystemVerilog segprm** (top-level entity) με τεχνολογία κοινής καθόδου.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα αρχείο **.do (segprm.do)**.
6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο **.do**, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.
7. Να προτείνετε με μια παραλλαγή της case την ελαχιστοποίηση του συστήματος.

Μέρος VI

Στο μέρος αυτό θα σχεδιάσουμε με μεικτό τρόπο (mixed), συνδυάζοντας περιγραφές ροής δεδομένων και ιεραρχικό/δομικό σχεδιασμό. Η σχεδίαση μας είναι ένας **4-bit πλήρης αθροιστής ριπής κρατουμένου (Full Adder Ripple Carry)**. Η λογική του 1-bit πλήρη αθροιστή, το σύμβολό τους αλλά και ο πίνακας αλήθειας και το σχέδιο του 4-bit πλήρη αθροιστή ριπής κρατουμένου παρουσιάζονται στην **Εικόνα 6**.



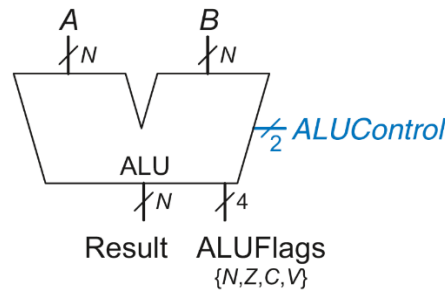
Εικόνα 6. Ο 4-bit πλήρης αθροιστής και τα υποκυκλώματά του.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

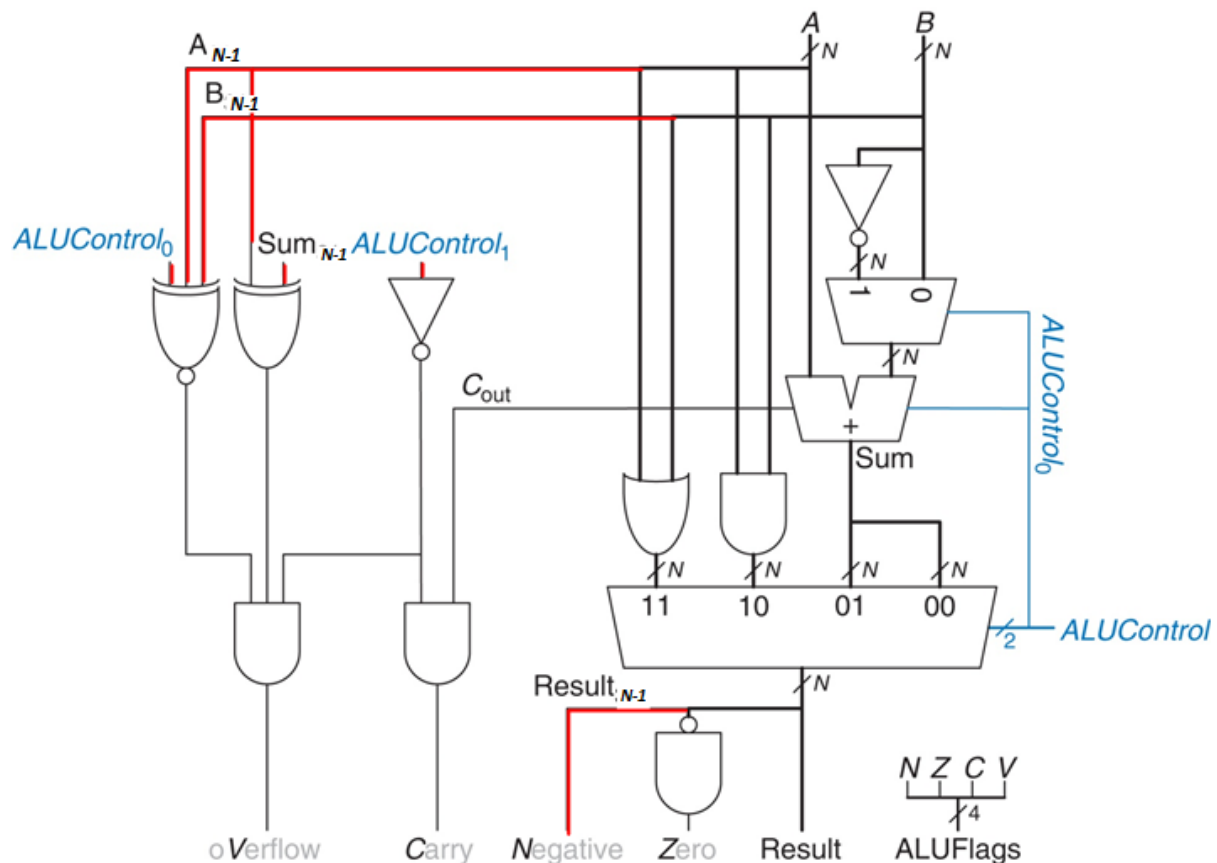
1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε την περιγραφή **SystemVerilog farc4bit** (top-level entity) και το υποκύκλωμα **fa1bit**.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα αρχείο .do (**farc4bit.do**).
6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Μέρος VII

Στο μέρος αυτό θα σχεδιάσετε μια **παραμετρική N-bit Αριθμητική – Λογική Μονάδα με Σημαίες Εξόδου (Status Flags - NZCV)**, όπως παρουσιάζεται στην **Εικόνα 7**. Οι είσοδοι και έξοδοι της σχεδίασης φαίνονται στο σχέδιο και τα σήματα ελέγχου **ALUControl** καθορίζουν τις πράξεις που θα εκτελεί το κύκλωμα. Στην Εικόνα 8 φαίνεται αναλυτικά η δομή της ALU και οι σημαίες Εξόδου.



Εικόνα 7. Το μπλοκ διάγραμμα της ALU N bit με Σημαίες Εξόδου.



Εικόνα 8. Το αναλυτικό δομικό της ALU N bit.

Τα τέσσερα bit των **ALUFlags** πρέπει να είναι TRUE αν η συνθήκη ικανοποιείται. Τα οποία είναι:

ALUFlags [3..0] bit	Περιγραφή
3	Το αποτέλεσμα είναι αρνητικό (N)
2	Το αποτέλεσμα είναι 0 (Z)
1	Ο αθροιστής παράγει carry out (C)
0	Ο αθροιστής παράγει overflow (V)

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

+1, AND, OR

1. Αναγνωρίστε την λειτουργία του κυκλώματος και τις πράξεις που υποστηρίζει και συμπληρώστε τον Πίνακα 1 με τις πράξεις για N=8 (οι αριθμοί είναι στο δεκαεξαδικό).
2. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε Board MAX 10 DE10 - Lite με το τσιπ προορισμού MAX 10 (10M50DAF484C6GES) και ορίστε ως προσομοιωτή τον Altera-ModelSim, με γλώσσα περιγραφής SystemVerilog HDL.
3. Δημιουργήστε την περιγραφή SystemVerilog alunbit.sv (top-level entity).
4. Υλοποιήστε το κύκλωμα σας (Analysis & Synthesis).
5. Καλέστε τον προσομοιωτή Altera-ModelSim για προσομοίωση RTL επιπέδου.
6. Με εντολές command line του Altera-ModelSim να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα αρχείο .do (alunbit.do).
7. Παραδοτέα: κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Πίνακας 1. Πράξεις ALU για N=8 (οι αριθμοί είναι hex).

Test	ALUControl[1:0]	A	B	Result	ALUFlags (NZCV)
ADD 0+0	0 0	00	00	00	4
ADD 0+(-1)	0 0	00	FF	FF	8
ADD 1+(-1)	0 0	01	FF	00	6
ADD FF+1	0 0	FF	01	00	6
SUB 0-0	0 1	00	00	00	6
SUB 0-(-1)	0 1	00	FF	01	0
SUB 1-1	0 1	01	01	00	4
SUB 80-1	0 1	80	01	7F	0
AND FF, FF	1 0	FF	FF	FF	10
AND FF, 78	1 0	FF	78	78	0
AND 78, 21	1 0	78	21	20	0
AND 00, FF	1 0	00	FF	00	4
OR FF, FF	1 1	FF	FF	FF	10
OR 78, 21	1 1	78	21	79	2
OR 00, FF	1 1	00	FF	FF	8
OR 00, 00	1 1	00	00	00	6

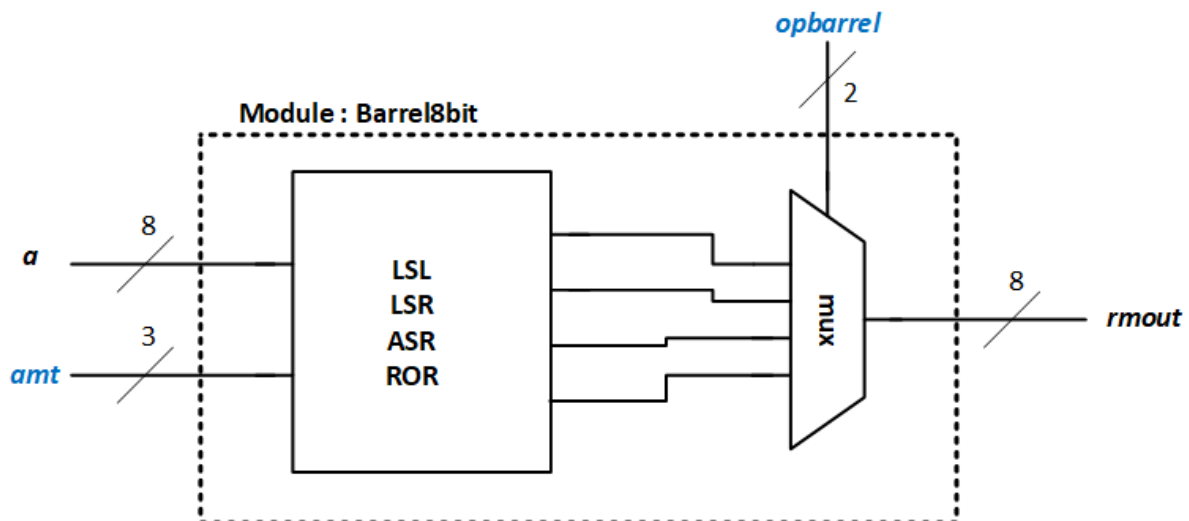
Μέρος VIII

Στο μέρος αυτό θα σχεδιάσετε έναν **ολισθητή παντός τύπου (Barrel Shifter)** των **8-bit** που θα εκτελεί τις παρακάτω τέσσερις πράξεις:

Πίνακας 2. Πράξεις Ολισθητή Barrel.

Πράξη	Κωδικός
LSL : Λογική ολίσθηση αριστερά	opbarrel (0,1)
LSR : Λογική ολίσθηση δεξιά	opbarrel (0,0)
ASR : Αριθμητική ολίσθηση δεξιά	opbarrel (1,1)
ROR : Κυκλική ολίσθηση δεξιά	opbarrel (1,0)

Το σύστημα που πρέπει να υλοποιήσετε παρουσιάζεται στην **Εικόνα 9**. Το σύστημα έχει μια είσοδο **a** που είναι 8 bit και είναι αυτή των δεδομένων, μια ακόμη είσοδο **amt** που έχει 3 bit και καθορίζει το πλήθος των ολισθήσεων (από 0 έως 7), και τέλος την είσοδο **opbarrel** που είναι 2 bit και επιλέγει μια από τις τέσσερις ολισθήσεις που προδιαγράφονται στον πίνακα 2.



Εικόνα 9. Η υλοποίηση του Barrel Shifter.

Παρακάτω σας δίνεται ένα δείγμα κώδικα που μπορείτε να χρησιμοποιήσετε για να υλοποιήσετε με παρόμοιο τρόπο και τις τέσσερις ολισθήσεις (ο δικός σας Barrel είναι των 8-bit).

```
// right rotated
always_comb
case (amt)
    2'b00: rotate_right = a;
    2'b01: rotate_right = {a[0], a[3], a[2], a[1]};
    2'b10: rotate_right = {a[1], a[0], a[3], a[2]};
    2'b11: rotate_right = {a[2], a[1], a[0], a[3]};
    default: rotate_right = 4'bxxxx;
endcase
```

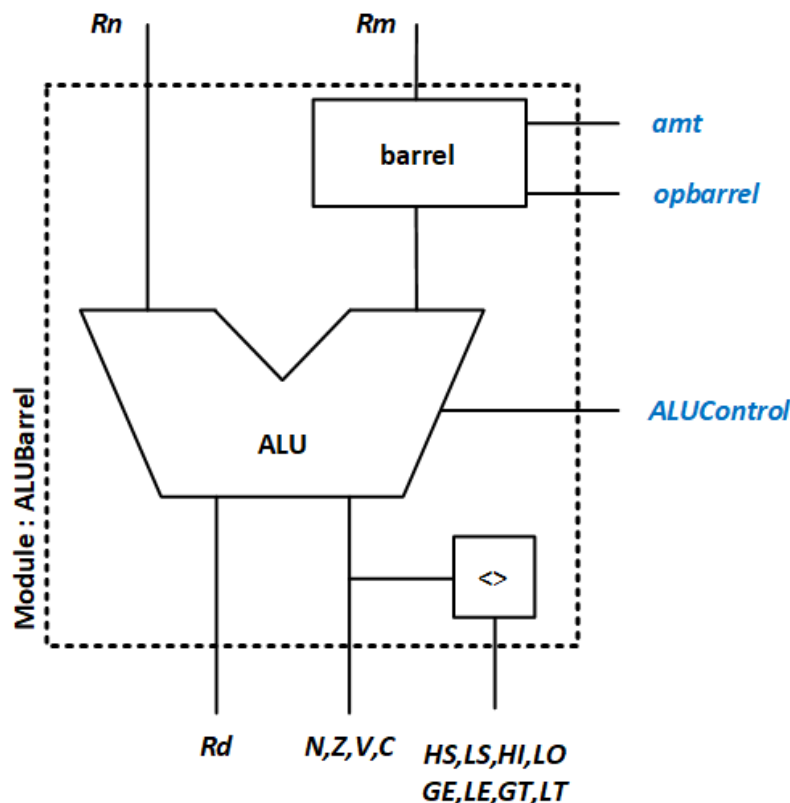
Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε την περιγραφή **SystemVerilog Barrel8bit.sv** (top-level entity).

3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα αρχείο .do (**Barrel8bit.do**).
6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Μέρος IX

Στο μέρος αυτό θα συνδέσετε τις υπομονάδες: της ALU (Μέρος VII), του Barrel (Μέρος VIII) και των CompFlags (Μέρος I) για να υλοποιήσετε το σύστημα που απεικονίζεται στην **Εικόνα 10**.



Εικόνα 10. Υπομονάδα Επεξεργαστή (ALU με ολισθητή παντός τύπου).

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε την περιγραφή **SystemVerilog ALUBarrel.sv** (top-level entity).
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Να αποθηκεύσετε τις εντολές σε ένα αρχείο .do (**ALUBarrel.do**).
6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

«ΠΡΟΗΓΜΕΝΕΣ ΕΦΑΡΜΟΓΕΣ ΨΗΦΙΑΚΗΣ ΣΧΕΔΙΑΣΗΣ»

Στο μέρος αυτό θα πρέπει να δώσετε ιδιαίτερη προσοχή στις εισόδους του αρχείου δοκιμής κατά την προσομοίωση (είναι σημαντικό για την βαθμολόγηση της άσκησης). Όταν έχουμε σύγκριση δύο αριθμών A και B (προσημασμένων ή μη προσημασμένων) η είσοδος στο υποσύστημα σύγκρισης είναι τα ALUFlags, ενώ η ALU εκτελεί την πράξη της αφαίρεσης $R_n - R_m$ (όπως έχουμε διδαχθεί στην Οργάνωση Υπολογιστών). Η άσκηση αυτή λύνεται για $N=8$.

ΚΑΛΗ ΕΠΙΤΥΧΙΑ!!!

ΠΑΡΑΡΤΗΜΑ

SystemVerilog Coding Styles

Ο τρόπος που περιγράφουμε μια σχεδίαση **SystemVerilog** έχει να κάνει και με το «στυλ» που υιοθετούμε στην συγγραφή και σηματοδοσία σε διάφορα σημεία του κώδικα.

Για το δικό μας εργαστήριο θα πρέπει να υιοθετήσουμε κάποιες συμβάσεις – προθέματα για να διευκολύνουμε την ζωή μας ως Σχεδιαστής Ψηφιακών Συστημάτων.

Υπάρχουν τρία βασικά οφέλη για την υιοθέτηση του στυλ συγγραφής κώδικα, αυτά είναι:

- ✓ Η υψηλή αναγνωσιμότητα του κώδικα. Ο κώδικας είναι εύκολα κατανοητός.
- ✓ Υψηλή συγκέντρωση στη συγγραφή του κώδικα.
- ✓ Ο κώδικας είναι λιγότερο επιρρεπής σε σφάλματα.

Περισσότερες πληροφορίες στον σύνδεσμο: <https://www.systemverilog.io/styleguide>

Προσοχή: Η SystemVerilog είναι case-sensitive!!!

Δήλωση της SystemVerilog ως γλώσσα στο Quartus Prime

Πηγαίνουμε στο μενού του Quartus Prime:

Assignments > Settings > Compiler Settings > Verilog HDL Input

Και επιλέγουμε: "**SystemVerilog**" under Verilog version.

BIBΛΙΟΓΡΑΦΙΑ - ΑΝΑΦΟΡΕΣ

Οι ασκήσεις βασίζονται σε εκπαιδευτικό υλικό από:

1. το ακαδημαϊκό σύγγραμμα «Ψηφιακή σχεδίαση και αρχιτεκτονική υπολογιστών - Έκδοση ARM» των S.Harris και D.Harris, Εκδόσεις ΚΛΕΙΔΑΡΙΘΜΟΣ.
2. Intel Corporation - FPGA University Program.

Πίνακας 3. Μνημονικά Συνθήκης.

Μνημονικό	Περιγραφή	Λογική
EQ	Equal (ίσοι)	Z
NE	Not equal (μη ίσοι)	$\bar{Z}$
CS / HS	Carry set / Unsigned higher or same (καθορισμός κρατουμένου / μη προσημασμένος υψηλότερος ή ίδιος)	C
CC / LO	Carry clear / Unsigned lower (επαναφορά κρατουμένου / μη προσημασμένος χαμηλότερος)	$\bar{C}$
MI	Minus / Negative (μείον / αρνητικό)	N
PL	Plus / Positive of zero	$\bar{N}$
VS	Overflow / Overflow set (υπερχείλιση / ενεργοποίηση υπερχείλισης)	V
VC	No overflow / Overflow clear (μη υπερχείλιση / επαναφορά υπερχείλισης)	$\bar{V}$
HI	Unsigned higher (μη προσημασμένος υψηλότερος)	$\bar{Z}C$
LS	Unsigned lower or same (μη προσημασμένος χαμηλότερος ή ίδιος)	$Z \text{ OR } \bar{C}$
GE	Signed greater than or equal (προσημασμένος μεγαλύτερος ή ίσος)	$\overline{N \oplus V}$
LT	Signed less than (προσημασμένος μικρότερος)	$N \oplus V$
GT	Signed greater than (προσημασμένος μεγαλύτερος)	$\bar{Z}(\overline{N \oplus V})$
LE	Signed less than or equal (προσημασμένος μικρότερος ή ίσος)	$Z \text{ OR } (N \oplus V)$
AL (ή none)	Always / unconditional (πάντα / χωρίς συνθήκη)	-

ModelSIM – Basic Commands

Βασικές εντολές του προσομοιωτή ModelSim που μπορούν να χρησιμοποιηθούν σε τερματικό παράθυρο (shell) ή στο παράθυρο εντολών (transcript window)

Εντολές δημιουργίας βιβλιοθήκης σχεδίασης και μεταγλώττισης

(Design library creation and design compilation commands)

vlib	-	Create a new design library
vmap	-	Define mapping between logical library name and directory path
vdir	-	List the contents of a design library
vdel	-	Delete a design unit from a specified library
vmake	-	Create a Makefile for a specified design library
vlog	-	Compile Verilog source code into a specified design library
vsim	-	Invoke the VSIM simulator
verror i	-	Prints info about warning/error messages, i is 4-digit msg number

Εντολές Προσομοίωσης (Simulation commands)

view	-	Create windows (wave, list, source etc).
add wave	-	Add objects to Wave window
force	-	Force a stimulus to Verilog signals
	>	Force input1 to 0 at the current simulator time.
		force input1 0
	>	Force the fourth element of the array bus1 to 1 at the current simulator time.
		force bus1(4) 1
	>	Force bus1 to 0110 at 100 nanoseconds after the current simulator time.
		force bus1 'b0110 100 ns
change	-	Change the value of a Verilog variable, constant or generic
run	-	Run simulation
restart	-	Reload and restart the simulation, option -f overrides need for confirmation
do	-	Execute a macro (do) file

Εντολές Αποσφαλμάτωσης (Debug commands)

examine	-	Examine the value of a signal or a variable
bp	-	Set a breakpoint
disablebp	-	Disable breakpoints
enablebp	-	Enable breakpoints
checkpoint	-	Save a simulator state
restore	-	Restore saved simulator state

Περισσότερα στον οδηγό του ModelSIM:

modelsim_ref.pdf (<φάκελος εγκατάστασης>\19.1\modelsim_ase\docs\pdfdocs)